

DESARROLLO DE SOFTWARE

Clase VI: Pruebas de Software

QA, tipos de testing, TDD y herramientas de automatización

2026



QA/QC

QUALITY ASSURANCE

VS

QUALITY CONTROL

Enfoque proactivo para **prevenir defectos** haciendo énfasis en el proceso usado en el desarrollo del producto, garantizando que lo propuesto en el plan de calidad se lleve a cabo.

Es una **actividad transversal** en el ciclo de vida del desarrollo del producto y se lleva a cabo mediante **auditorías de calidad regulares** con el objetivo de identificar oportunamente las desviaciones entre lo planeado y lo realizado.

¿Quién es responsable del QA?

Cada miembro del **equipo involucrado en el desarrollo del producto** es responsable. Un ejemplo de actividad de QA en el desarrollo de software es la **verificación de especificaciones técnicas**.



"QA asegura la calidad de los procesos utilizados para crear productos de calidad"

Proceso reactivo puntual, enfocado en **identificar y corregir** los defectos encontrados en el producto ya terminado antes de ser liberado o entregado al cliente final.

¿Quién es responsable del QC?

Usualmente es realizado por un **equipo de trabajo independiente** que se encuentra exclusivamente dedicado a probar el producto para encontrar errores. Un ejemplo de actividad de QC en el desarrollo de software es la **validación funcional**.

"QC es el proceso de probar la calidad de un producto"



El término de QA es muy utilizado entre las compañías del sector servicios con una gran presencia en el mundo tecnológico.

Aunque el término "QA" es relativamente nuevo, ya que el mundo de "testing", "automatización de pruebas", etc., encontró salida en el mercado IT a mediados del año 2008, muchísima gente confunde control de calidad (QC) con el término QA (Quality Assurance), que en sus siglas en Inglés, viene a significar GARANTÍA DE CALIDAD.

Los nombres son muy parecidos pero se refieren a procesos totalmente distintos, si bien es verdad que el QA se basa mucho en el feedback que se recibe desde QC, aumentando aún más la confusión.

QA vs QC — GARANTÍA VS. CONTROL DE CALIDAD

Característica	QA (Quality Assurance)	QC (Quality Control)
Foco	Prevención de defectos (proceso)	Detección de defectos (producto)
¿Cuándo actúa?	Desde el inicio del proyecto	Cuando el producto está terminado
¿Qué hace?	Define estándares y parámetros de aceptación	Controla el comportamiento del producto final
¿Quién trabaja con?	Dev, ingenieros, managers, cliente	Con el equipo de QA
Orientación	Al proceso	Al producto
Ejemplo	Definir criterios de aceptación antes de desarrollar	Ejecutar pruebas sobre el sistema finalizado

PRUEBAS DE SOFTWARE

Las pruebas de software son cruciales para garantizar la calidad y confiabilidad del software. Ayudan a detectar errores tempranamente, prevenir problemas y mejorar la experiencia del usuario.

Las pruebas de software son un conjunto de actividades y procesos que se realizan para evaluar la calidad de un software.

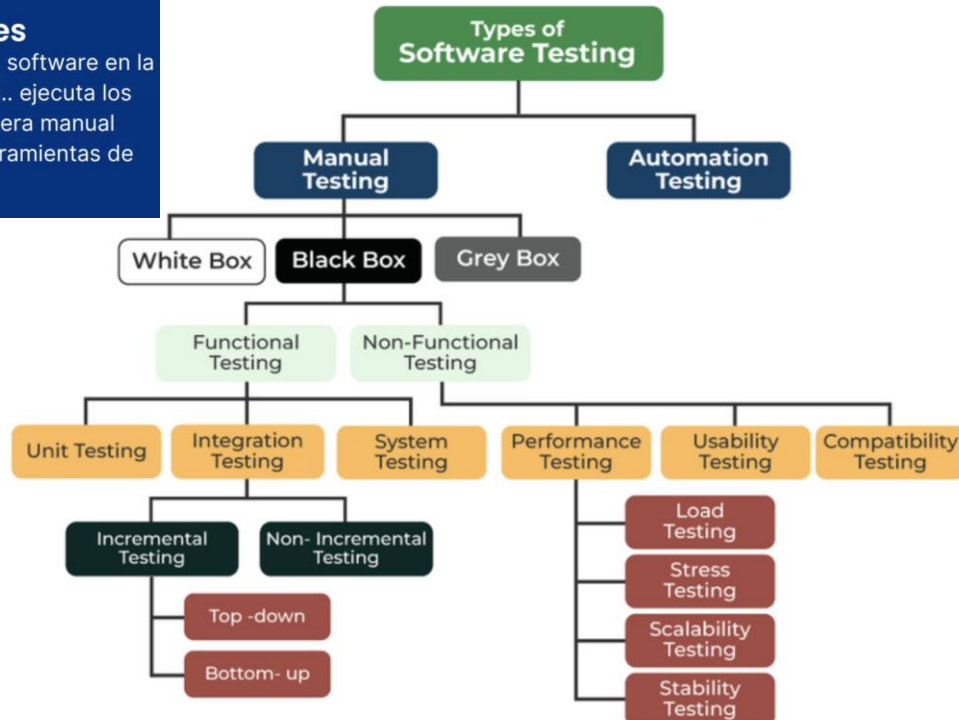
Su objetivo es identificar y corregir errores, verificar que el software cumple con los requisitos especificados y garantizar que funcione de manera correcta y confiable.

Las pruebas de software son una parte integral del desarrollo de software y se realizan en diferentes etapas del ciclo de vida del software.

TIPOS DE PRUEBAS DE SOFTWARE

Pruebas Manuales

Son un tipo de prueba de software en la cual el tester/analista/etc.. ejecuta los casos de prueba de manera manual excluyendo el uso de herramientas de automatización.



Pruebas Automatizadas

Es una técnica de pruebas que consiste en utilizar scripts y herramientas para controlar la ejecución de las pruebas y la comparación de los resultados reales con los resultados esperados.

Pruebas Automatizadas



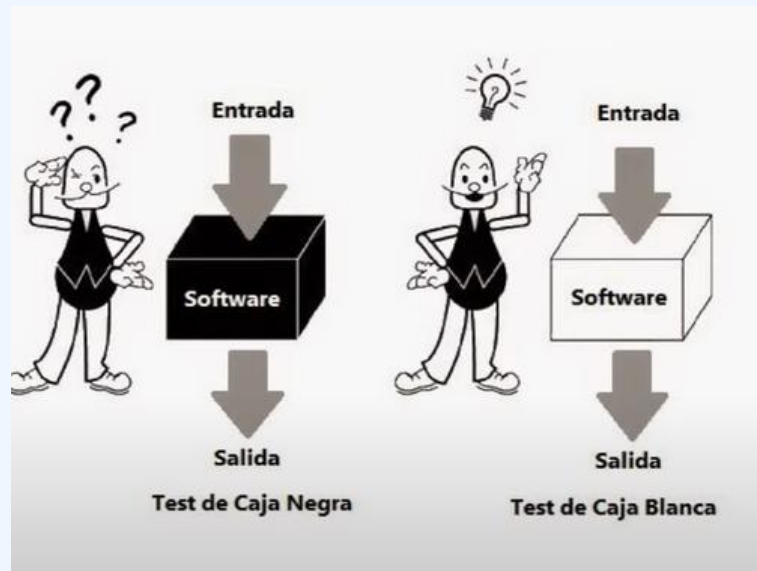
TIPOS DE PRUEBAS DE SOFTWARE

Caja Negra (Black-Box): Se centra en las entradas y salidas, validando los requisitos funcionales desde la perspectiva del usuario final sin conocer la estructura interna.

Es ideal para pruebas funcionales y de sistema.

Caja Blanca (White-Box): Requiere conocimientos de programación para analizar el flujo de datos, estructuras lógicas y el código fuente. Es común en pruebas unitarias y de integración para detectar errores ocultos.

Caja Gris (Gray-Box): Un enfoque híbrido donde el tester tiene acceso limitado a la arquitectura, base de datos o documentación técnica, permitiendo realizar pruebas más enfocadas que la caja negra.



TIPOS DE PRUEBAS DE SOFTWARE

Pruebas Unitarias

Prueban unidades individuales de código (funciones, clases). Aisladas y rápidas. Ej: JUnit, PyTest.

Pruebas de Integración

Verifican que módulos separados funcionen correctamente juntos. Detectan problemas de interfaz.

Pruebas de Sistema

Evalúan el sistema completo en su conjunto. Verifican flujos de negocio completos.

Pruebas de Aceptación (UAT)

El usuario final valida que el sistema cumple sus necesidades reales.
Determinan si el software cumple con los requisitos definidos por los usuarios o clientes.

Pruebas de Caja Negra

Sin conocer el código interno. Solo entradas y salidas esperadas. Perspectiva de usuario.

Pruebas de Caja Blanca

Conociendo el código. Se diseñan casos según rutas de ejecución. Requiere cobertura de sentencias.

CASO DE PRUEBA

Un caso de prueba es un conjunto de acciones que se realizan en un sistema para determinar si cumple con los requisitos de software y funciona correctamente.

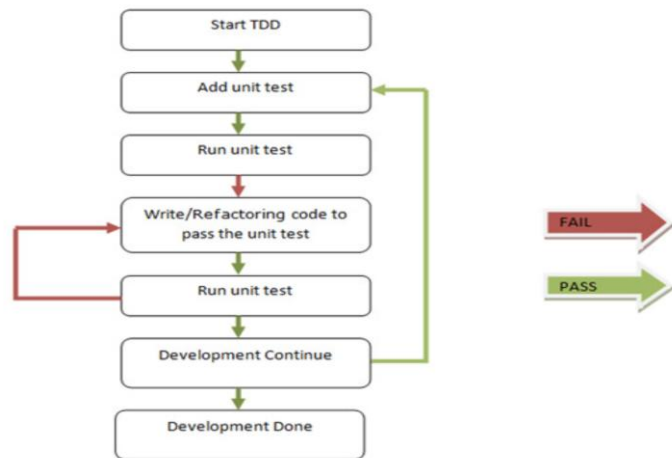
Su propósito es verificar que las diferentes funcionalidades del sistema se comporten como se espera y confirmar que el sistema cumpla con todos los estándares, directrices y requisitos del cliente.

El proceso de escribir un caso de prueba también puede ayudar a detectar errores o defectos en el sistema.

ID de Caso de Prueba	Identificador único del caso de prueba
Nombre de Caso de prueba	Caso Uso aspecto a Probar
Descripción	Se probará la respuesta del sistema cuando se presenta X escenario...
Precondiciones	Condiciones que se deben cumplir para la ejecución del caso de prueba.
Relaciones Casos de uso	Indicar si el caso de uso al cual se le está derivando los casos de prueba tiene conexiones con casos de uso mediante relaciones de include, extend o generalización.
Pasos y condiciones ejecución	Pasos detallados del caso de prueba y los datos con los que se probará
Resultado Esperado	Resultado ideal de la aplicación de acuerdo a los pasos ejecutados
Estado Caso de Prueba	• Ejecutado ○ Exitoso ○ Fallido ○ Frenado • Pendiente de ejecución • En construcción
Resultado Obtenido	Se diligencia luego de la ejecución del caso de prueba y de acuerdo a la reacción de la aplicación
Errores Asociados	Al ejecutar el caso de prueba, en este campo se indican los identificadores únicos de los errores presentados.
Responsable Diseño	Nombre del analista de pruebas que diligenció la plantilla
Responsable Ejecución	Nombre del tester que ejecutó el caso de prueba
Comentarios	

TDD — TEST DRIVEN DEVELOPMENT

¿Qué es Test Driven Development? El test driven development (TDD) o en español desarrollo guiado por pruebas, es un enfoque de programación que se utiliza durante el desarrollo de software en el que se realizan pruebas unitarias antes de escribir el código.



- Hacer el test garantiza la cobertura y el correcto diseño.
- Detectar casos de borde.
- El developer está obligado a hacer el test.
- Complicado para legacy.
- Se tiene que entender el requerimiento.
- Verificar tema costos y tiempos.

TDD — TEST DRIVEN DEVELOPMENT

TDD invierte el orden clásico: primero se escribe el test, luego el código que lo hace pasar. Garantiza cobertura y mejor diseño desde el inicio.

1. RED

Escribir un test que falla (porque el código no existe aún).

2. GREEN

Escribir el código mínimo necesario para que el test pase.

3. REFACTOR

Mejorar el código sin romper los tests. Limpiar y optimizar.

Ventajas: cobertura garantizada, mejor diseño, documentación viva. Desventaja: lento al inicio, difícil en legacy.

HERRAMIENTAS DE TESTING

JUnit / TestNG

Unit Testing Java

Frameworks estándar para pruebas unitarias en Java. Integración total con Maven/Gradle y CI/CD.

PyTest / Unittest

Unit Testing Python

PyTest es el más popular para Python. Fixtures, parametrización y plugins para cobertura.

Selenium / Playwright

Automatización Web

Controlan el browser para simular acciones del usuario. Playwright es la evolución moderna de Selenium.

Postman / Newman

Pruebas de API

Postman para pruebas manuales de APIs REST/GraphQL. Newman para ejecutar colecciones en CI/CD.

SonarQube

Análisis de Código

Detecta bugs, code smells y vulnerabilidades de seguridad. Muestra porcentaje de cobertura de tests.

Appium

Testing Móvil

Automatización de pruebas en apps iOS y Android. Compatible con código nativo, híbrido y web.

IA EN PRUEBAS DE SOFTWARE — 2026

La IA está revolucionando el testing: desde la generación automática de casos de prueba hasta la detección predictiva de defectos antes de que ocurran.

♦ Generación de test cases

Copilot y Claude generan automáticamente pruebas unitarias y de integración para funciones existentes, cubriendo edge cases.

♦ Testing visual con IA

Applitools y Percy usan IA para detectar diferencias visuales en UI automáticamente, reemplazando capturas manuales.

♦ Análisis predictivo de bugs

Herramientas como Diffblue Cover analizan el código y predicen qué funciones tienen mayor probabilidad de tener defectos.

♦ Self-healing tests

Los tests de automatización se 'auto-reparan' cuando la UI cambia, sin necesidad de actualización manual.

CONCLUSIÓN

Las pruebas de software son esenciales para desarrollar software de alta calidad y confiable.

Al realizar pruebas de manera regular, se pueden detectar y corregir errores tempranamente, prevenir problemas y mejorar la experiencia del usuario.

Existen diferentes tipos de pruebas de software, cada una con su propio enfoque y objetivo.

TDD (Test Driven Development) es una metodología de desarrollo de software que promueve la escritura de pruebas unitarias antes de escribir el código.

Existen diversas herramientas de testing que facilitan la realización de pruebas de software.

¡MUCHAS GRACIAS!

Clase VI: Pruebas de Software

Próxima clase: 12-05 VIRTUAL 14:30 + REVISIÓN POR GRUPOS .